

**LAPORAN PRAKTIKUM
SISTEM MULTIMEDIA**

**MODUL 2
DESIGN PRINCIPLE (S.O.L.I.D)**

DISUSUN OLEH :

AMMAR BAGAS FATHURRAHMAN WANTORO 2350081008



**PROGRAM STUDI INFORMATIKA
FAKULTAS SAINS DAN INFORMATIKA
UNIVERSITAS JENDERAL ACHMAD YANI
TAHUN 2025**

DAFTAR ISI

DAFTAR ISI	1
DAFTAR GAMBAR	Error! Bookmark not defined.
BAB I. HASIL PRAKTIKUM	2
I.1 Latihan SRP	2
I.1.A. Source Code	2
I.1.B. Analisa	4
I.2 Latihan OCP	5
I.2.A. Source code	5
I.2.B. Analisa	8
I.3 Latihan LSP	8
I.3.A. Source code	8
I.3.B. Analisa	11
BAB II. TUGAS PRAKTIKUM	12
II.1 Tugas	12
II.1.A. Soal	12
II.1.B. Jawaban	12
BAB III. KESIMPULAN	14

BAB I. HASIL PRAKTIKUM

I.1 Latihan SRP

I.1.A. Source Code

A) Class -class sebelum SRP

```
package Latihan_SRP;

public class Order {
    void calculateTotalSum() {

    }
    void.getItems() {

    }
    void getItemCount() {

    }
    void addItem(Item item) {

    }
    void deleteItem(Item item) {

    }

    void printOrder() {

    }
    void showOrders() {

    }

    void getDailyHistory() {
    }
    void getMonthHistory() {
    }
}
```

B) Class - class sesudah SRP

```
public class Order {  
    void calculateTotalSum() {  
  
    }  
    void.getItems() {  
  
    }  
    void getItemCount() {  
  
    }  
    void addItem(Item item) {  
  
    }  
    void deleteItem(Item item) {  
  
    }  
}
```

```
package Latihan_SRP;  
  
public class OrderHistory {  
    void getDailyHistory() {  
  
    }  
    void getMonthlyHistory() {  
  
    }  
}
```

```
package Latihan_SRP;  
  
public class OrderViewer {  
    void printOrder(Order order) {}  
    void showOrder(Order order) {}  
}
```

```
package Latihan_SRP;

public class Main {
    public static void main(String[] args) {
        Item item = new Item();
        Order order = new Order();
        order.addItem(item);
        OrderHistory history = new OrderHistory();
        history.getDailyHistory();
        OrderViewer viewer = new OrderViewer();
        viewer.printOrder(order);
    }
}
```

I.1.B. Analisa

Sebelum penerapan SRP, kelas Order melanggar prinsip tersebut dengan menanggung terlalu banyak tanggung jawab, mulai dari logika bisnis inti (kalkulasi, manajemen item) hingga urusan tampilan (printOrder) dan akses data histori (getDailyHistory). Hal ini menciptakan High Coupling (ketergantungan tinggi) dan Low Cohesion (kohesi rendah), di mana semua fungsi saling terkait erat. Akibatnya, kode menjadi kaku, sulit diuji, dan sangat berisiko untuk dimodifikasi; perubahan kecil pada format tampilan saja bisa berpotensi merusak logika bisnis yang krusial. Setelah menerapkan SRP, tanggung jawab tersebut dipecah menjadi kelas-kelas yang lebih fokus: Order hanya mengurus logika bisnis, OrderViewer mengurus tampilan, dan OrderHistory mengurus pengambilan data. Setiap kelas kini memiliki High Cohesion dan satu alasan tunggal untuk berubah. Pemecahan ini menghasilkan kode yang Low Coupling (fleksibel), jauh lebih mudah diuji secara terisolasi, aman untuk dikembangkan, dan komponennya (seperti Order) dapat digunakan kembali dengan lebih mudah dalam konteks yang berbeda.

I.2 Latihan OCP

I.2.A. Source code

A) Sebelum menerapkan OCP

```
package Latihan_OpenClose;

public class Cinema {
    public Double price;
}
```

```
package Latihan_OpenClose;

public class StandardCimena extends Cinema{
    public StandardCimena(double price){
        this.price = price;
    }
}
```

```
package Latihan_OpenClose;

public class DeluxeCinema extends Cinema{
    public DeluxeCinema(double price){
        this.price = price;
    }
}
```

```
package Latihan_OpenClose;

public class PremiumCinema extends Cinema {
    public PremiumCinema(double price){
        this.price = price;
    }
}
```

```

package Latihan_OpenClose;

public class CinemaCalculations {
    public Double calculateAdminFee(Cinema cinema){
        if(cinema instanceof StandardCimena){
            return ((StandardCimena)cinema).price;
        }else if(cinema instanceof DeluxeCinema){
            return ((DeluxeCinema)cinema).price;
        }else if (cinema instanceof PremiumCinema){
            return ((PremiumCinema)cinema).price;
        }else{
            return 0.0;
        }
    }
}

```

B) Dengan penerapan OCP

```

package Latihan_OpenClose;

public abstract class Cinema1 {
    public Double price;
    abstract Double calculateAdminFee();
}

```

```

package Latihan_OpenClose;

public class StandardCimena extends Cinema1{
    public StandardCimena(double price){
        this.price = price;
    }

    @Override
    Double calculateAdminFee() {
        return price * 10 / 100;
    }
}

```

```
package Latihan_OpenClose;

public class DeluxeCinema extends Cinema1{
    public DeluxeCinema(double price){
        this.price = price;
    }

    @Override
    Double calculateAdminFee() {
        return price * 12/100;
    }
}
```

```
package Latihan_OpenClose;

public class PremiumCinema extends Cinema1 {
    public PremiumCinema(double price){
        this.price = price;
    }

    @Override
    Double calculateAdminFee() {
        return price * 20/100;
    }
}
```

```
package Latihan_OpenClose;

public class Main {
    public static void main(String[] args) {
        StandardCimena standardCimena = new StandardCimena(100.0);
        Double adminFee = standardCimena.calculateAdminFee();
        System.out.println(adminFee);
    }
}
```


I.2.B. Analisa

Kode "sebelum" secara jelas melanggar Open/Closed Principle (OCP) karena kelas CinemaCalculations tidak "tertutup" untuk modifikasi. Jika ada penambahan tipe bioskop baru (misalnya IMAXCinema), programmer dipaksa untuk mengubah kode yang sudah ada di dalam kelas CinemaCalculations dengan menambahkan blok else if baru. Penggunaan instanceof dan struktur if-else yang panjang ini adalah tanda utama pelanggaran OCP, yang membuat sistem menjadi kaku, rapuh, dan sulit untuk dipelihara seiring bertambahnya fungsionalitas.

Kode "sesudah" berhasil menerapkan OCP dengan baik menggunakan polimorfisme. Dengan mengubah Cinema menjadi abstract class (Cinema1) dan mendefinisikan abstract method calculateAdminFee(), tanggung jawab kalkulasi didelegasikan ke masing-masing subclass. Sekarang, sistem "terbuka" untuk ekstensi—kita bisa menambahkan tipe bioskop baru (seperti IMAXCinema) hanya dengan membuat kelas baru yang meng-implementasikan metode calculateAdminFee(). Yang terpenting, sistem "tertutup" untuk modifikasi, karena tidak ada kode yang sudah ada (seperti kelas StandardCinema atau DeluxeCinema) yang perlu diubah saat fungsionalitas baru ditambahkan.

I.3 Latihan LSP

I.3.A. Source code

A) Sebelum menerapkan LSP

```
package Latihan_LSP;

import java.util.Date;

public abstract class Product {
    abstract String getName();
    abstract Date getExpiredDate();

    public void getProductInfo() {
        //some magic code
    }
}
```

```
package Latihan_LSP;

import java.util.Date;

public class Vegetable extends Product{
    @Override
    String getName() {
        return "Broccolli";
    }

    @Override
    Date getExpirationDate() {
        return new Date();
    }
}
```

```
package Latihan_LSP;

import java.util.Date;

public class Smartphone extends Product {
    @Override
    String getName() {
        return "Samsung S10+ limited edition";
    }

    @Override
    Date getExpirationDate() {
        return new Date();
    }
}
```

B) Sesudah menerapkan LSP

```
package Latihan_LSP;

public abstract class Product1 {
    abstract String getName();
    public void getProduction() {

    }
}
```

```
package Latihan_LSP;

import java.util.Date;

public abstract class FoodProduct extends Product1 {
    abstract Date getExpirationDate();
}
```

```
package Latihan_LSP;

import java.util.Date;

public class Vegetable extends FoodProduct{
    @Override
    String getName() {
        return "Broccolli";
    }

    @Override
    Date getExpirationDate() {
        return new Date();
    }
}
```

```
package Latihan_LSP;

import java.util.Date;

public class Smartphone extends Product1 {
    @Override
    String getName() {
        return "Samsung S10+ limited edition";
    }
}
```

I.3.B. Analisa

Kode "sebelum" melanggar Liskov Substitution Principle (LSP) karena parent class `Product` memaksakan abstraksi yang salah pada semua turunannya. Ia mendeklarasikan abstract method `getExpiredDate()`, yang memaksa semua subclass untuk mengimplementasikannya. Meskipun ini logis untuk `Vegetable`, ini sama sekali tidak relevan untuk `Smartphone`. Akibatnya, kelas `Smartphone` dipaksa untuk menyediakan implementasi "palsu" atau tidak bermakna (seperti `return new Date()`), yang bisa membingungkan dan menyebabkan bug. Ini melanggar LSP karena objek `Smartphone` tidak dapat sepenuhnya disubstitusikan di tempat yang mengharapkan `Product` (terutama yang mengharapkan tanggal kedaluwarsa) tanpa mengubah perilaku program.

Kode "sesudah" memperbaiki pelanggaran ini dengan menerapkan hierarki warisan yang benar. Metode `getExpirationDate()` yang spesifik telah dihilangkan dari `Product1` (kelas basis umum) dan dipindahkan ke abstract class perantara baru, yaitu `FoodProduct`. Dengan cara ini, `Product1` hanya berisi metode yang berlaku untuk semua produk (seperti `getName()`). Kelas `Vegetable` sekarang mewarisi `FoodProduct` (karena memang logis), sementara `Smartphone` hanya mewarisi `Product1`. Desain ini mematuhi LSP karena setiap subclass sekarang hanya mewarisi metode dan perilaku yang relevan dengannya, memastikan bahwa objek `Vegetable` atau `Smartphone` dapat menggantikan parent class mereka dengan aman tanpa menimbulkan error atau perilaku tak terduga.

BAB II. TUGAS PRAKTIKUM

II.1 Tugas

II.1.A. Soal

1. Pada Latihan 3.2.1 Single Responsibility, Jelaskan kenapa bentuk/design codenya harus diubah ?
2. Pada Latihan 3.2.2 Open/Close Principle, Jelaskan alasan kenapa bentuk/design codenya harus diubah ?
3. Buatla Class CinemaMahasiswa yang biaya adminya adalah 5% dari harga umum
4. Pada Latihan 3.2.3 Liskov Substitution, Jelaskan alasan kenapa bentuk/design codenya harus diubah ?

II.1.B. Jawaban

1. Desain kode Order yang lama wajib diubah karena melanggar SRP dengan menjadi God Class yang mencampurkan tiga tanggung jawab: logika bisnis, logika tampilan, dan logika akses data. Pencampuran ini sangat berbahaya karena menciptakan high coupling, di mana perubahan sepele pada fungsi tampilan (seperti printOrder) dapat secara tidak sengaja merusak logika bisnis inti (seperti calculateTotalSum). Selain itu, kelas ini menjadi kaku karena memiliki terlalu banyak alasan untuk berubah, sulit diuji secara terisolasi, dan tidak bisa digunakan kembali di platform lain, sehingga pemisahan menjadi Order, OrderViewer, dan OrderHistory mutlak diperlukan untuk stabilitas dan pemeliharaan kode
2. Desain kode CinemaCalculations yang lama harus diubah karena secara fundamental melanggar Open/Closed Principle. Kelas tersebut tidak "tertutup" untuk modifikasi, setiap kali ada tipe bioskop baru (misalnya IMAX), programmer dipaksa untuk membuka dan mengubah kode yang sudah ada di dalam CinemaCalculations dengan menambahkan blok else if baru. Penggunaan instanceof ini menciptakan sistem yang kaku (rigid) dan rapuh (fragile), di mana satu kelas pusat harus diubah terus-menerus dan berisiko tinggi merusak fungsionalitas yang sudah ada, padahal seharusnya sistem cukup "terbuka" untuk diekstensi dengan menambahkan kelas baru tanpa mengubah kode lama.

3. Membuat Class CinemaMahasiswa

```
package Latihan_OpenClose;

public class CinemaMahasiswa extends Cinema1{
    public CinemaMahasiswa(double price){ this.price = price; }

    @Override
    Double calculateAdminFee() {
        return price * 5/100;
    }
}
```

4. Desain kode "sebelum" harus diubah karena abstraksi yang salah tersebut secara fundamental melanggar Liskov Substitution Principle (LSP). Kelas Product memaksa semua turunannya, termasuk Smartphone, untuk mengimplementasikan `getExpiredDate()`, sebuah metode yang sama sekali tidak relevan bagi smartphone. Ini memaksa Smartphone untuk memberikan implementasi "palsu" atau tidak bermakna, yang berarti objek Smartphone tidak dapat lagi disubstitusikan dengan aman untuk objek Product di setiap konteks—terutama jika konteks itu bergantung pada tanggal kedaluwarsa—tanpa berisiko menimbulkan bug atau mengubah perilaku program secara tak terduga.

BAB III. KESIMPULAN

Praktikum implementasi Design Principle ini telah mendemonstrasikan secara nyata bagaimana penerapan Single Responsibility Principle (SRP), Open/Closed Principle (OCP), dan Liskov Substitution Principle (LSP) secara fundamental meningkatkan kualitas desain perangkat lunak. Melalui studi kasus, terlihat jelas bahwa kode yang tidak menerapkan prinsip-prinsip ini (seperti pada kelas Order yang God Class, CinemaCalculations yang kaku, dan hierarki Product yang salah) cenderung sangat rapuh, sulit dimodifikasi, dan sulit untuk diuji.

Penerapan SRP berhasil memisahkan kelas berdasarkan tanggung jawabnya, menghasilkan kode yang high cohesion dan low coupling. OCP membimbing penggunaan polimorfisme untuk menciptakan sistem yang fleksibel dan dapat diperluas tanpa harus mengubah kode yang sudah ada dan stabil. Terakhir, LSP memastikan bahwa hierarki pewarisan dibangun dengan benar secara logis, sehingga substitusi objek subclass tidak merusak perilaku program. Secara keseluruhan, ketiga prinsip ini adalah pilar penting untuk membangun perangkat lunak yang maintainable (mudah dipelihara), scalable (mudah dikembangkan), dan robust (andal).